Mobile application for iOS

PINGLISH

1. Project Introduction

Pinglish is an iOS app designed for language learning through short, interactive lessons, with a focus on a young audience.
The app allows users to create an account, log in, complete lessons, track their progress, and compete on a points-based leaderboard.
This project has been developed using Swift and SwiftUI, following the Model-View-ViewModel architectural pattern to separate the concepts, testability and maintainability.

2. Project Structure

The project is organized in the following way:

2.1 Models

Models represent the core data structures of the application.

→ AppUser: Represents a registered user.
The main attributes are: unique identifier (UUID), username and points accumulated
The model conforms to Codable to allow easy persistence. In this way, when a young user reopens the app, they'll find all their progress saved exactly where they left off, without having to start from scratch.

→ Course: Represents a language course (e.g., Spanish, Italian, German).

→ Level: Groups lessons by topic.

→ Lesson: Represents an individual learning activity.
Lessons include reference word, sentence with a missing word, correct answer, an associated image and a completion state.

→ LeaderboardEntry: Represents an entry in the leaderboard, storing: User identifier, username and points.

2.2 ViewModels

ViewModels manage application logic and state, they use @Published properties and conform to ObservableObject to enable reactive UI updates via SwiftUI.

→ AuthViewModel: Responsible for user authentication.
Main responsibilities are user registration (sign-up), login validation, logout handling and the current user state management.

User credentials are securely stored using Keychain Services, while user metadata is stored in UserDefaults.

→ CoursesViewModel: Manages courses and lesson progress.
The main responsibilities are loading sample course data, persisting lesson completion per user, isolating course progress by username and updating lesson completion state safely.
Courses are stored per user, ensuring that each user has independent progress.

→ LeaderboardViewModel: Manages the leaderboard system.
The main responsibilities are to add or update user scores, sort leaderboard entries by points, persist leaderboard data and provide a clean interface for both UI and testing.
A configurable initializer is used to allow test isolation.

→ SettingsViewModel: The SettingsViewModel manages application configuration and user preferences. It acts as the logical layer between the settings user interface and persistent storage.

ViewModels use @Published properties and conform to ObservableObject to enable reactive UI updates via SwiftUI.

2.3 MusicManager

The application includes a dedicated component called MusicManager, responsible for managing background audio playback within the app.
MusicManager is designed to play background music during application usage, ensure a consistent audio experience across different views and prevent multiple audio instances from playing simultaneously.

2.4 Views

Views are implemented using SwiftUI and are responsible only for presentation. Views observe ViewModels and react automatically to state changes.

3. Data Persistence

The project uses multiple persistence mechanisms:

- UserDefaults: Used for storing lightweight structured data such as courses, lesson progress, and leaderboard entries.
- Keychain Services: Used to securely store user passwords.

Each user has isolated stored data, ensuring progress and scores are user-specific.

4. Architecture Pattern (MVVM)

The application follows the Model–View–ViewModel (MVVM) pattern:

- Model: Defines data structures.
- View: Displays UI elements and observes ViewModels.

- ViewModel: Contains logic, state, and data manipulation.

5. Future Improvements

There are several areas where future improvements could be implemented in order to enhance scalability and user experience. Currently, the application relies on UserDefaults and Keychain for local data persistence, which is an appropriate choice for a lightweight project and small-scale data storage. However, for a production-level application or scenarios involving larger datasets, a migration to a database-based solution such as Core Data or SQLite would be advisable.

6. Conclusion

This project demonstrates the design and implementation of a structured iOS application following the MVVM architectural pattern. Throughout the development process, particular emphasis has been placed on separation of concerns, clean state management, and clear responsibilities between views, view models, and services.

Furthermore, the incorporation of persistent storage mechanisms highlights the project's focus on reliability, which are key aspects of professional software development. Overall, the application serves as a solid foundation that not only fulfills its current requirements but is also well-prepared for future enhancements.